

İkinci El Araç Fiyat Tahmini — Regresyon Projesi

Veri seti: `cars.csv` — Türkiye ikinci el otomobil ilan verileri (~52 400 satır, 23 sütun)

Hedef değişken: `fiyat` (TL cinsinden sürekli değer)

Proje adımları:

1. Keşifsel Veri Analizi (EDA)
2. Veri Temizleme ve Ayrıştırma (Parse)
3. Özellik Mühendisliği
4. Train / Test Ayrımı
5. Ön-işleme Pipeline'i
6. Baseline ve Gelişmiş Modeller
7. Değerlendirme (MAE, RMSE, R²)
8. Özellik Önemi ve Hata Analizi

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler, OrdinalEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Ridge, ElasticNet
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score
import warnings, re

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", 30)
print("Kütüphaneler yüklendi.")
```

Kütüphaneler yüklendi.

1. Veri Yükleme ve İlk Bakış

```
In [2]: df = pd.read_csv("cars.csv", encoding="utf-8")
print(f"Boyut: {df.shape[0]} satır × {df.shape[1]} sütun")
df.head(3)
```

Boyut: 52256 satır × 23 sütun

Out[2]:	baslik	konum	fiyat	ilan_tarihi	marka	seri	model	yil	kilometre	vites_tipi	yakit_tipi	kasa_tipi	ren
0	2022 MEGANE 1.3 TCE 140 HP JOY COMFORT EDC SER...	Yeni Mh. Sarıçam, Adana	1.169.000 TL	21 Ağustos 2025	Renault	Megane	1.3 Tce Joy Comfort	2022	124.000 km	Otomatik	Benzin	Sedan	Beya
1	2022 SKODA OCTAVIA 1.0 E-TEC ELITE DSG SERVIS ...	Yeni Mh. Sarıçam, Adana	1.520.000 TL	21 Ağustos 2025	Skoda	Octavia	1.0 e- Tec Elite	2022	99.000 km	Otomatik	Hibrit	Sedan	G
2	Sahibinden HATASIZ BOYASIZ Peugeot 207	Cemalpaşa Mh. Seyhan, Adana	469.999 TL	19 Ağustos 2025	Peugeot	207	1.4 Trendy	2009	207.500 km	Düz	LPG & Benzin	Hatchback/5	G

```
In [3]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52256 entries, 0 to 52255
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   baslik                 52256 non-null  object
1   konum                  52256 non-null  object
2   fiyat                  52256 non-null  object
3   ilan_tarihi            52256 non-null  object
4   marka                  52256 non-null  object
5   seri                   52256 non-null  object
6   model                  52185 non-null  object
7   yil                    52256 non-null  int64
8   kilometre              52255 non-null  object
9   vites_tipi             52244 non-null  object
10  yakit_tipi              52256 non-null  object
11  kasa_tipi                52252 non-null  object
12  renk                    52254 non-null  object
13  motor_hacmi             51103 non-null  object
14  motor_gucu              50895 non-null  object
15  cekis                   51163 non-null  object
16  arac_durumu             52250 non-null  object
17  ortalama_yakit_tuketimi 31568 non-null  object
18  yakit_deposu            37211 non-null  object
19  boya_degisen            52256 non-null  object
20  takasa_uygun            34612 non-null  object
21  kimden                  52256 non-null  object
22  tramer                  5424 non-null   float64
dtypes: float64(1), int64(1), object(21)
memory usage: 9.2+ MB

```

2. Sütun Bazlı Profil: Tip, Örnek Değer, Eksik %, Benzersiz Sayı

```

In [4]: profile = pd.DataFrame({
        "dtype": df.dtypes,
        "non_null": df.notnull().sum(),
        "null_count": df.isnull().sum(),
        "null_pct": (df.isnull().sum() / len(df) * 100).round(2),
        "nunique": df.nunique(),
        "sample_value": df.iloc[0].values,
    })
profile

```

Out[4]:

	dtype	non_null	null_count	null_pct	nunique	sample_value
baslik	object	52256	0	0.00	37537	2022 MEGANE 1.3 TCE 140 HP JOY COMFORT EDC SER...
konum	object	52256	0	0.00	9174	Yeni Mh. Sarıçam, Adana
fiyat	object	52256	0	0.00	3951	1.169.000 TL
ilan_tarihi	object	52256	0	0.00	161	21 Ağustos 2025
marka	object	52256	0	0.00	59	Renault
seri	object	52256	0	0.00	423	Megane
model	object	52185	71	0.14	2951	1.3 TCe Joy Comfort
yil	int64	52256	0	0.00	54	2022
kilometre	object	52255	1	0.00	4802	124.000 km
vites_tipi	object	52244	12	0.02	3	Otomatik
yakit_tipi	object	52256	0	0.00	5	Benzin
kasa_tipi	object	52252	4	0.01	11	Sedan
renk	object	52254	2	0.00	24	Beyaz
motor_hacmi	object	51103	1153	2.21	254	1201 - 1400 cm3
motor_gucu	object	50895	1361	2.60	203	126 - 150 HP
cekis	object	51163	1093	2.09	5	Önden Çekiş
arac_durumu	object	52250	6	0.01	4	İkinci El
ortalama_yakit_tuketimi	object	31568	20688	39.59	89	NaN
yakit_deposu	object	37211	15045	28.79	51	NaN
boya_degisen	object	52256	0	0.00	88	1 değişen
takasa_uygun	object	34612	17644	33.76	3	Takasa Uygun
kimden	object	52256	0	0.00	3	Galeriden
tramer	float64	5424	46832	89.62	1465	71300.0

In [5]:

```
print("=== Kategorik sütunlardaki benzersiz değerler (ilk 10) ===\n")
cat_cols = ["marka", "seri", "vites_tipi", "yakit_tipi", "kasa_tipi",
            "renk", "cekis", "arac_durumu", "takasa_uygun", "kimden"]
for col in cat_cols:
    vc = df[col].value_counts().head(10)
    print(f"--- {col} ({df[col].nunique()} benzersiz) ---")
    print(vc.to_string(), "\n")
```

=== Kategorik sütunlardaki benzersiz değerler (ilk 10) ===

--- marka (59 benzersiz) ---

```
marka
Renault      8563
Volkswagen   6185
Fiat          5633
Opel          5284
Ford          3756
Hyundai      2900
Toyota       2737
Peugeot      2010
BMW          1955
Mercedes - Benz 1848
```

--- seri (423 benzersiz) ---

```
seri
Astra      2686
Clio       2461
Megane     2243
Corolla    2129
Passat     2065
Focus      2035
Egea       1874
Polo       1492
Corsa      1421
Civic      1329
```

--- vites_tipi (3 benzersiz) ---

```
vites_tipi
Düz        33003
Otomatik   15351
Yarı Otomatik 3890
```

--- yakit_tipi (5 benzersiz) ---

yakit_tipi	
Benzin	18709
Dizel	18522
LPG & Benzin	14417
Hibrit	397
Elektrik	211

--- kasa_tipi (11 benzersiz) ---

kasa_tipi	
Sedan	29787
Hatchback/5	18763
Station wagon	1275
Hatchback/3	905
MPV	718
Coupe	610
-	64
SUV	64
Cabrio	49
Roadster	13

--- renk (24 benzersiz) ---

renk	
Beyaz	19400
Gri	8412
Siyah	5672
Gri (Gümüş)	3286
Kırmızı	2884
Mavi	2564
Füme	2149
Lacivert	1343
Bordo	1135
Gri (metalik)	1077

--- cekis (5 benzersiz) ---

cekis	
Önden Çekiş	45335
Arkadan İtiş	4825
4WD (Sürekli)	880
-	117
AWD (Elektronik)	6

--- arac_durumu (4 benzersiz) ---

arac_durumu	
İkinci El	52245
Yetkili Bayiden Sıfır	2
Yurtdışından İthal Sıfır	2
Sıfır	1

--- takasa_uygun (3 benzersiz) ---

takasa_uygun	
Takasa Uygun	23443
Takasa Uygun Değil	10696
-	473

--- kimden (3 benzersiz) ---

kimden	
Sahibinden	29137
Galeriden	22858
Yetkili Bayiden	261

```
In [6]: print("=== Sayısal / parse gerektiren sütunlar – örnek değerler ===\n")
for col in ["fiyat", "kilometre", "motor_hacmi", "motor_gucu",
            "ortalama_yakit_tuketimi", "yakit_deposu", "tramer", "boya_degisen"]:
    print(f"--- {col} ---")
    print(df[col].dropna().sample(min(8, df[col].dropna().shape[0]), random_state=42).values, "\n")
```

=== Sayısal / parse gerektiren sütunlar – örnek değerler ===

```
--- fiyat ---
['750.000 TL' '559.000 TL' '305.000 TL' '466.500 TL' '610.000 TL'
 '415.000 TL' '5.400.000 TL' '500.000 TL']

--- kilometre ---
['248.000 km' '215.000 km' '182.000 km' '326.000 km' '246.000 km'
 '240.000 km' '347.000 km' '188.000 km']

--- motor_hacmi ---
['1368 cc' '1596 cc' '1500 cc' '1401 - 1600 cm3' '1401 - 1600 cm3'
 '1401 - 1600 cm3' '1598 cc' '1390 cc']

--- motor_gucu ---
['125 hp' '60 hp' '126 - 150 HP' '76 - 100 HP' '76 - 100 HP'
 '101 - 125 HP' '90 hp' '122 hp']

--- ortalama_yakit_tuketimi ---
['3,4 lt' '4,9 lt' '7,7 lt' '5,6 lt' '7,7 lt' '4,7 lt' '6 lt' '5,1 lt']

--- yakit_deposu ---
['60 lt' '42 lt' '52 lt' '45 lt' '42 lt' '45 lt' '45 lt' '70 lt']

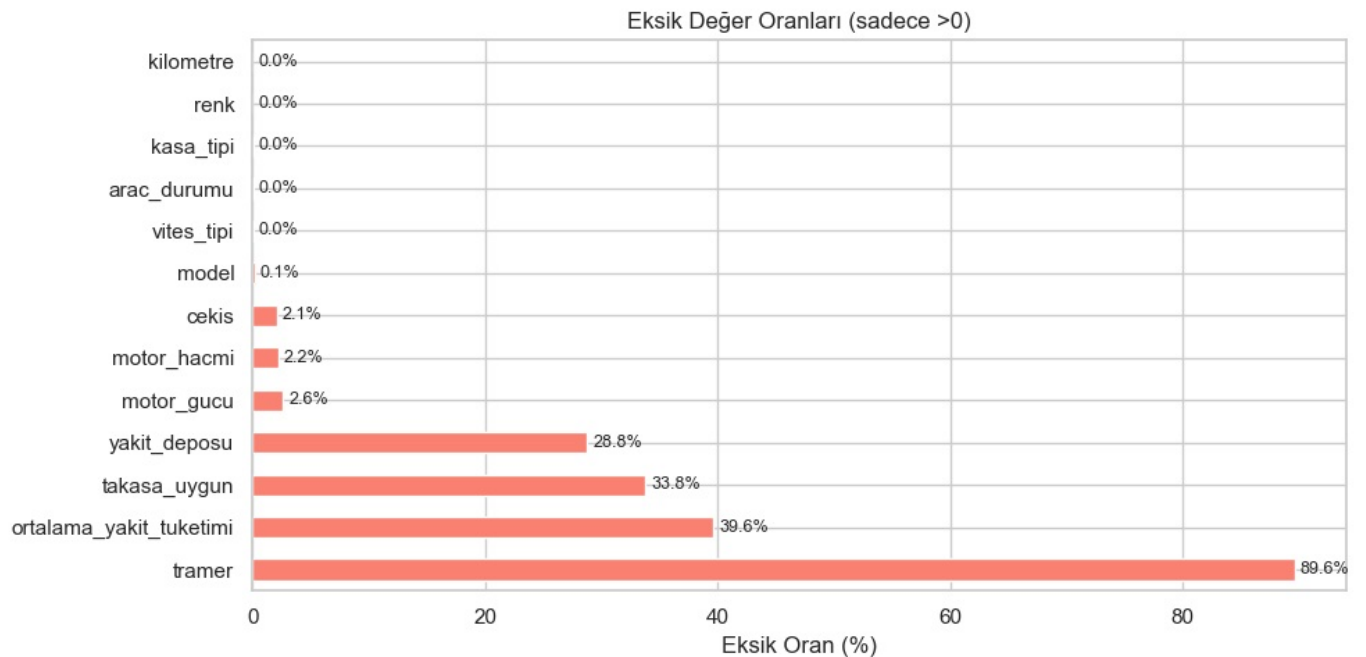
--- tramer ---
[6.20000e+03 3.18400e+03 3.75000e+05 8.20000e+01 4.30000e+02 7.10565e+05
 7.00000e+00 4.93300e+03]

--- boya_degisen ---
['2 boyalı' '2 boyalı' '4 boyalı' '1 değişen, 9 boyalı' 'Tamamı orjinal'
 '3 boyalı' 'Tamamı orjinal' 'Belirtilmemiş']
```

3. Eksik Veri Haritası

```
In [7]: null_pct = (df.isnull().sum() / len(df) * 100).sort_values(ascending=False)
null_pct = null_pct[null_pct > 0]

fig, ax = plt.subplots(figsize=(10, 5))
null_pct.plot.barh(ax=ax, color="salmon")
ax.set_xlabel("Eksik Oran (%)")
ax.set_title("Eksik Değer Oranları (sadece >0)")
for i, v in enumerate(null_pct.values):
    ax.text(v + 0.5, i, f"{v:.1f}%", va="center", fontsize=9)
plt.tight_layout()
plt.show()
```



```
In [8]: dup_count = df.duplicated(subset=["marka", "seri", "yil", "kilometre", "fiyat"]).sum()
print(f"Olası yinelenen ilan sayısı (marka+seri+yıl+km+fiyat eşleşmesi): {dup_count}")
print(f"Toplam satır: {len(df)}")
print(f"Yinelenen oran: %{dup_count / len(df) * 100:.2f}")
```

Olası yinelenen ilan sayısı (marka+seri+yıl+km+fiyat eşleşmesi): 378
Toplam satır: 52256
Yinelenen oran: %0.72

4. Veri Temizleme ve Ayrıştırma (Parse)

Sayısal olması gereken ama string halinde bulunan sütunları temizliyoruz:

- fiyat : 1.169.000 TL → 1169000
- kilometre : 124.000 km → 124000
- motor_hacmi : 1368 cc veya 1201 - 1400 cm3 → orta değer
- motor_gucu : 91 hp veya 76 - 100 HP → orta değer
- ortalama_yakit_tuketimi : 6,3 lt → 6.3
- yakit_deposu : 50 lt → 50
- boya_degisen : boyalı ve değişen parça sayısı çıkarılacak

```
In [9]: def parse_fiyat(val):
        """1.169.000 TL' → 1169000"""
        if pd.isna(val):
            return np.nan
        s = str(val).replace(".", "").replace("TL", "").strip()
        try:
            return int(s)
        except ValueError:
            return np.nan

def parse_km(val):
        """124.000 km' → 124000"""
        if pd.isna(val):
            return np.nan
        s = str(val).replace(".", "").replace("km", "").strip()
        try:
            return int(s)
        except ValueError:
            return np.nan

def parse_motor_hacmi(val):
        """1368 cc' → 1368 veya '1201 - 1400 cm3' → 1300.5 (orta değer)"""
        if pd.isna(val):
            return np.nan
        s = str(val).strip()
        nums = re.findall(r"\d+", s)
        if len(nums) == 2:
            return (int(nums[0]) + int(nums[1])) / 2
        elif len(nums) == 1:
            return float(nums[0])
        return np.nan

def parse_motor_gucu(val):
        """91 hp' → 91 veya '76 - 100 HP' → 88"""
        if pd.isna(val):
            return np.nan
        s = str(val).strip()
        nums = re.findall(r"\d+", s)
        if len(nums) == 2:
            return (int(nums[0]) + int(nums[1])) / 2
        elif len(nums) == 1:
            return float(nums[0])
        return np.nan

def parse_yakit_tuketim(val):
        """6,3 lt' → 6.3"""
        if pd.isna(val):
            return np.nan
        s = str(val).replace(",", ".", "").replace("lt", "").strip()
        try:
            return float(s)
        except ValueError:
            return np.nan

def parse_yakit_deposu(val):
        """50 lt' → 50"""
        if pd.isna(val):
            return np.nan
        s = str(val).replace("lt", "").strip()
        try:
            return int(s)
        except ValueError:
            return np.nan

def parse_boya_degisen(val):
        """Boyalı ve değişen parça sayısını çıkarır. 'Tamamı orjinal' → (0,0)"""
        boyali, degisen = 0, 0
        if pd.isna(val):
```

```

        return np.nan, np.nan
    s = str(val).lower().strip()
    if "tamamı orjinal" in s:
        return 0, 0
    if "tamamı boyalı" in s:
        return 10, 0 # tüm paneller
    if "belirtilmemiş" in s:
        return np.nan, np.nan

    m_boyali = re.search(r"(\d+)\s*boyalı", s)
    m_degisen = re.search(r"(\d+)\s*değişen", s)
    if m_boyali:
        boyali = int(m_boyali.group(1))
    if m_degisen:
        degisen = int(m_degisen.group(1))
    return boyali, degisen

print("Parse fonksiyonları tanımlandı.")

```

Parse fonksiyonları tanımlandı.

```

In [10]: df["fiyat_num"] = df["fiyat"].apply(parse_fiyat)
df["km_num"] = df["kilometre"].apply(parse_km)
df["motor_hacmi_num"] = df["motor_hacmi"].apply(parse_motor_hacmi)
df["motor_gucu_num"] = df["motor_gucu"].apply(parse_motor_gucu)
df["yakit_tuketim_num"] = df["ortalama yakit_tuketimi"].apply(parse_yakit_tuketim)
df["yakit_deposu_num"] = df["yakit_deposu"].apply(parse_yakit_deposu)

boya_parsed = df["boya_degisen"].apply(parse_boya_degisen)
df["boyali_sayi"] = boya_parsed.apply(lambda x: x[0])
df["degisen_sayi"] = boya_parsed.apply(lambda x: x[1])

df["tramer_num"] = df["tramer"].fillna(0)

print("Parse sonrası yeni sütunlar:")
print(df[["fiyat_num", "km_num", "motor_hacmi_num", "motor_gucu_num",
          "yakit_tuketim_num", "yakit_deposu_num", "boyali_sayi",
          "degisen_sayi", "tramer_num"]].describe().round(2))

```

Parse sonrası yeni sütunlar:

	fiyat_num	km_num	motor_hacmi_num	motor_gucu_num	\
count	5.225600e+04	52255.00	40300.00	50777.00	
mean	7.621392e+05	207655.47	1456.83	106.91	
std	2.186182e+06	523298.10	300.88	37.20	
min	1.000000e+04	0.00	601.50	41.00	
25%	3.500000e+05	130000.00	1364.00	88.00	
50%	5.782500e+05	200000.00	1490.00	100.00	
75%	9.450000e+05	266000.00	1597.00	115.00	
max	4.670000e+08	99999999.00	6592.00	761.00	

	yakit_tuketim_num	yakit_deposu_num	boyali_sayi	degisen_sayi	\
count	31568.00	37211.00	41708.00	41708.00	
mean	5.72	52.28	2.61	0.54	
std	1.36	7.55	3.30	0.91	
min	0.80	30.00	0.00	0.00	
25%	4.50	45.00	0.00	0.00	
50%	5.70	50.00	1.00	0.00	
75%	6.70	55.00	4.00	1.00	
max	14.10	100.00	12.00	11.00	

	tramer_num
count	52256.00
mean	26429.45
std	723638.52
min	0.00
25%	0.00
50%	0.00
75%	0.00
max	31012019.00

```

In [11]: print(f"Parse sonrası eksik sayıları:")
for col in ["fiyat_num", "km_num", "motor_hacmi_num", "motor_gucu_num",
            "yakit_tuketim_num", "yakit_deposu_num", "boyali_sayi", "degisen_sayi"]:
    n = df[col].isna().sum()
    print(f" {col}: {n} ({n/len(df)*100:.2f}%)")

```

Parse sonrası eksik sayıları:

```

fiyat_num: 0 (0.00%)
km_num: 1 (0.00%)
motor_hacmi_num: 11956 (22.88%)
motor_gucu_num: 1479 (2.83%)
yakit_tuketim_num: 20688 (39.59%)
yakit_deposu_num: 15045 (28.79%)
boyali_sayi: 10548 (20.19%)
degisen_sayi: 10548 (20.19%)

```

5. Parse Sonrası Dağılımlar ve Görselleştirme

```
In [12]: fig, axes = plt.subplots(2, 3, figsize=(16, 10))

axes[0, 0].hist(df["fiyat_num"].dropna(), bins=80, color="steelblue", edgecolor="white")
axes[0, 0].set_title("Fiyat Dağılımı (TL)")
axes[0, 0].set_xlabel("Fiyat")

axes[0, 1].hist(np.log1p(df["fiyat_num"].dropna()), bins=80, color="darkorange", edgecolor="white")
axes[0, 1].set_title("log(Fiyat) Dağılımı")
axes[0, 1].set_xlabel("log(Fiyat)")

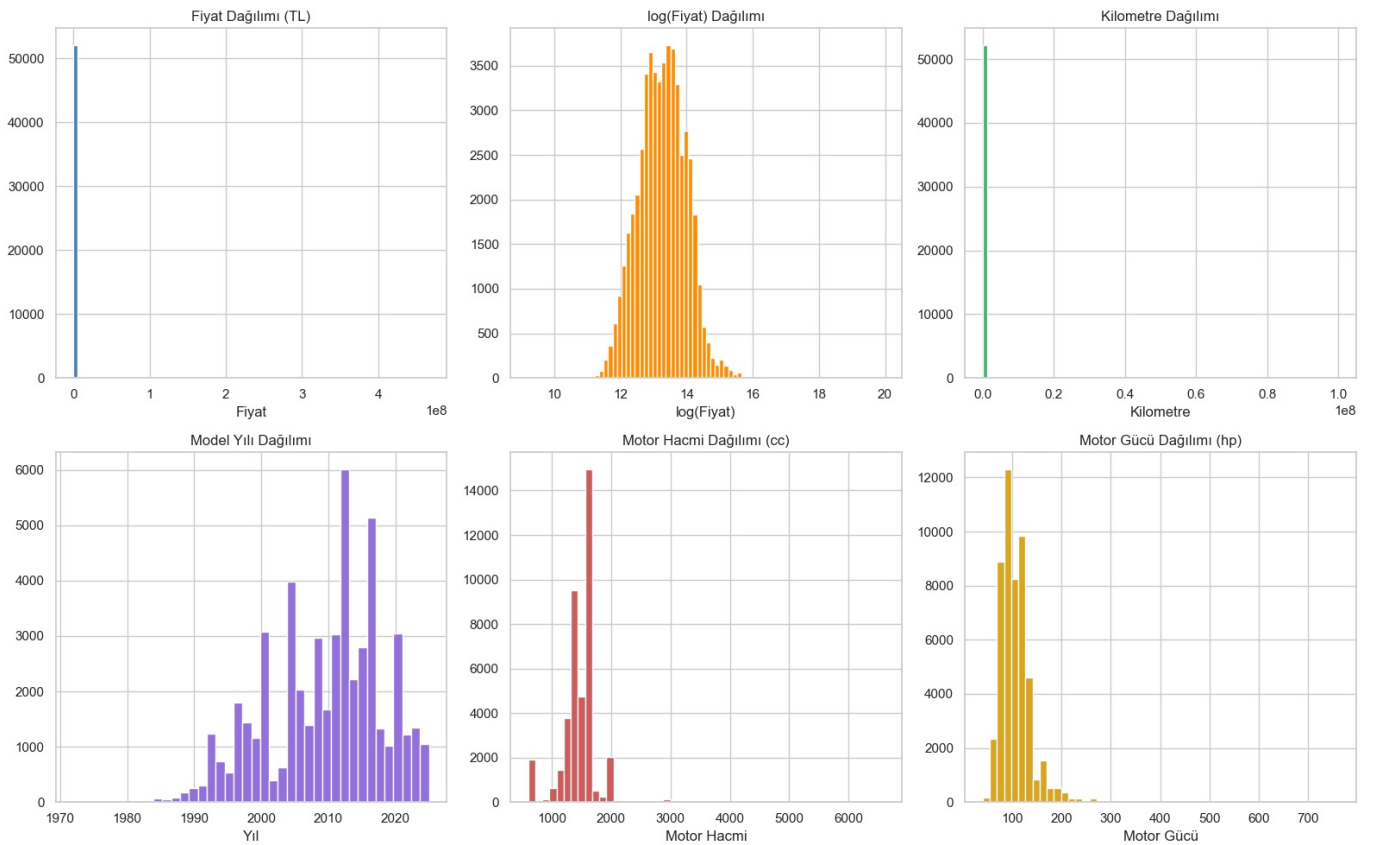
axes[0, 2].hist(df["km_num"].dropna(), bins=80, color="mediumseagreen", edgecolor="white")
axes[0, 2].set_title("Kilometre Dağılımı")
axes[0, 2].set_xlabel("Kilometre")

axes[1, 0].hist(df["yıl"].dropna(), bins=40, color="mediumpurple", edgecolor="white")
axes[1, 0].set_title("Model Yılı Dağılımı")
axes[1, 0].set_xlabel("Yıl")

axes[1, 1].hist(df["motor_hacmi_num"].dropna(), bins=50, color="indianred", edgecolor="white")
axes[1, 1].set_title("Motor Hacmi Dağılımı (cc)")
axes[1, 1].set_xlabel("Motor Hacmi")

axes[1, 2].hist(df["motor_gucu_num"].dropna(), bins=50, color="goldenrod", edgecolor="white")
axes[1, 2].set_title("Motor Gücü Dağılımı (hp)")
axes[1, 2].set_xlabel("Motor Gücü")

plt.tight_layout()
plt.show()
```



```
In [13]: fig, axes = plt.subplots(1, 3, figsize=(16, 5))

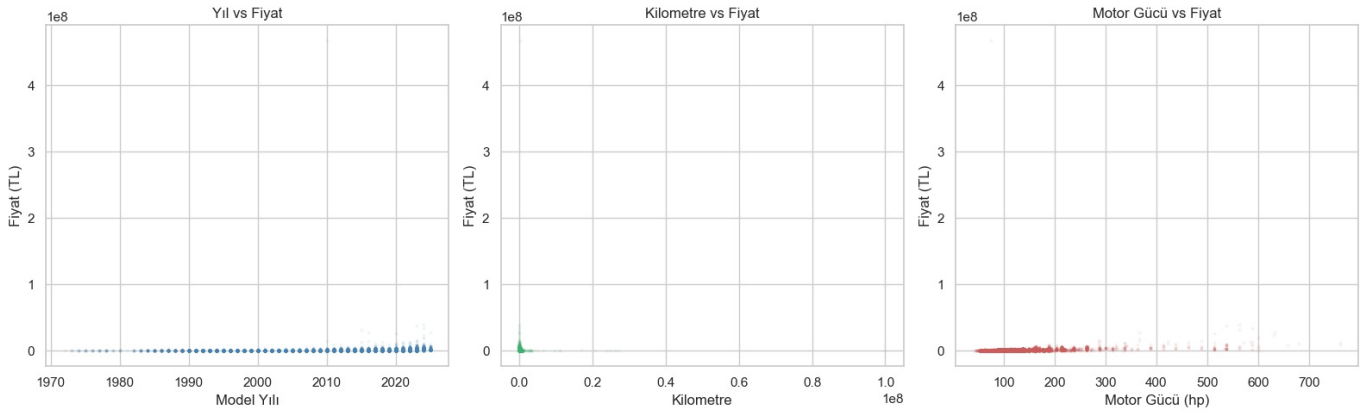
axes[0].scatter(df["yıl"], df["fiyat_num"], alpha=0.05, s=3, color="steelblue")
axes[0].set_xlabel("Model Yılı")
axes[0].set_ylabel("Fiyat (TL)")
axes[0].set_title("Yıl vs Fiyat")

axes[1].scatter(df["km_num"], df["fiyat_num"], alpha=0.05, s=3, color="mediumseagreen")
axes[1].set_xlabel("Kilometre")
axes[1].set_ylabel("Fiyat (TL)")
axes[1].set_title("Kilometre vs Fiyat")

axes[2].scatter(df["motor_gucu_num"], df["fiyat_num"], alpha=0.05, s=3, color="indianred")
axes[2].set_xlabel("Motor Gücü (hp)")
axes[2].set_ylabel("Fiyat (TL)")
axes[2].set_title("Motor Gücü vs Fiyat")
```

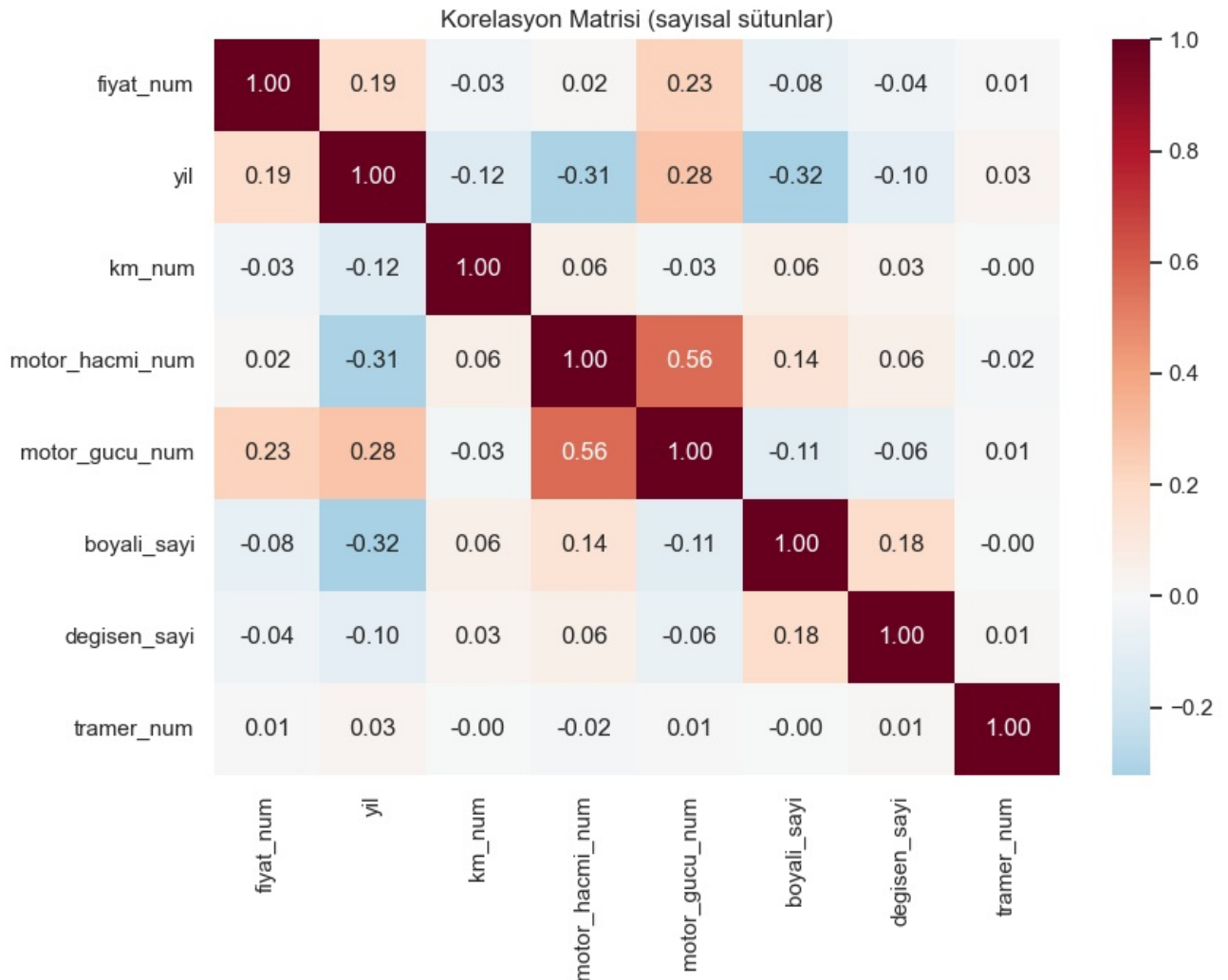


```
plt.tight_layout()
plt.show()
```



```
In [14]: num_cols = ["fiyat_num", "yil", "km_num", "motor_hacmi_num", "motor_gucu_num",
                    "boyali_sayi", "degisen_sayi", "tramer_num"]
corr = df[num_cols].corr()

fig, ax = plt.subplots(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0, ax=ax)
ax.set_title("Korelasyon Matrisi (sayısal sütunlar)")
plt.tight_layout()
plt.show()
```



```
In [15]: fig, axes = plt.subplots(1, 3, figsize=(16, 5))

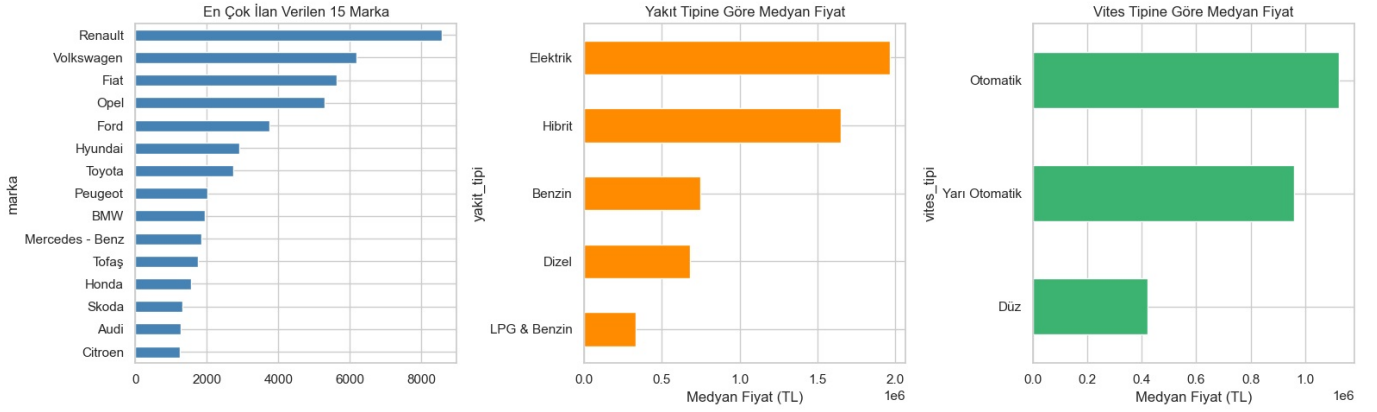
top_marka = df["marka"].value_counts().head(15)
top_marka.plot.barh(ax=axes[0], color="steelblue")
axes[0].set_title("En Çok İlan Verilen 15 Marka")
axes[0].invert_yaxis()

df.groupby("yakit_tipi")["fiyat_num"].median().sort_values().plot.barh(ax=axes[1], color="darkorange")
axes[1].set_title("Yakıt Tipine Göre Medyan Fiyat")
axes[1].set_xlabel("Medyan Fiyat (TL)")

df.groupby("vites_tipi")["fiyat_num"].median().sort_values().plot.barh(ax=axes[2], color="mediumseagreen")
axes[2].set_title("Vites Tipine Göre Medyan Fiyat")
```

```
axes[2].set_xlabel("Medyan Fiyat (TL)")
```

```
plt.tight_layout()  
plt.show()
```



6. Özellik Seçimi ve Temiz Veri Seti Oluşturma

Modelleme için kullanılacak sütunlar:

Sayısal: yıl, km_num, motor_hacmi_num, motor_gucu_num, boyali_sayi, degisen_sayi, tramer_num

Kategorik: marka, vites_tipi, yakit_tipi, kasa_tipi, cekis, kimden

Hariç tutulanlar:

- baslik : sızıntı riski + yüksek kardinalite
- konum : 9000+ benzersiz — ileri aşamada "il" çıkarılabilir
- seri, model : yüksek kardinalite; marka yeterli baseline için
- arac_durumu : neredeyse tamamı "İkinci El"
- ortalama_yakit_tuketimi, yakit_deposu : %30-40 eksik
- takasa_uygun : %34 eksik
- ilan_tarihi, renk : ileri aşama iyileştirme için bırakıldı

```
In [16]: numeric_features = ["yil", "km_num", "motor_hacmi_num", "motor_gucu_num",  
                           "boyali_sayi", "degisen_sayi", "tramer_num"]  
categorical_features = ["marka", "vites_tipi", "yakit_tipi", "kasa_tipi", "cekis", "kimden"]  
target = "fiyat_num"  
  
keep_cols = numeric_features + categorical_features + [target]  
dfm = df[keep_cols].copy()  
  
print(f"Seçilen sütunlarla boyut: {dfm.shape}")  
print(f"Hedef (fiyat_num) eksik: {dfm[target].isna().sum()}")  
  
dfm = dfm.dropna(subset=[target])  
dfm = dfm[dfm[target] > 0]  
print(f"Hedef NaN/0 temizliği sonrası: {dfm.shape[0]} satır")  
  
q_low = dfm[target].quantile(0.01)  
q_high = dfm[target].quantile(0.99)  
before = len(dfm)  
dfm = dfm[(dfm[target] >= q_low) & (dfm[target] <= q_high)]  
print(f"Fiyat outlier temizliği (P1-P99): {before} → {len(dfm)} satır")  
print(f"Fiyat aralığı: {dfm[target].min():.0f} – {dfm[target].max():.0f} TL")  
  
cat_dash_cols = ["cekis", "kasa_tipi"]  
for col in cat_dash_cols:  
    dfm[col] = dfm[col].replace("-", np.nan)  
  
print(f"\nSon boyut: {dfm.shape}")  
dfm.head()
```

Seçilen sütunlarla boyut: (52256, 14)

Hedef (fiyat_num) eksik: 0

Hedef NaN/0 temizliği sonrası: 52256 satır

Fiyat outlier temizliği (P1-P99): 52256 → 51260 satır

Fiyat aralığı: 120,000 – 3,397,500 TL

Son boyut: (51260, 14)

Out[16]:	yil	km_num	motor_hacmi_num	motor_gucu_num	boyali_sayi	degisen_sayi	tramer_num	marka	vites_tipi	yakit_tipi	k
0	2022	124000.0	NaN	138.0	0.0	1.0	71300.0	Renault	Otomatik	Benzin	
1	2022	99000.0	NaN	113.0	1.0	0.0	56060.0	Skoda	Otomatik	Hibrit	
2	2009	207500.0	1360.0	91.0	0.0	0.0	0.0	Peugeot	Düz	LPG & Benzin	Hatı
3	2013	219000.0	NaN	188.0	4.0	1.0	0.0	Audi	Otomatik	Dizel	
4	2013	335000.0	NaN	188.0	0.0	0.0	0.0	BMW	Otomatik	Benzin	

7. Train / Test Ayrımı

```
In [17]: X = dfm[numeric_features + categorical_features]
y = dfm[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Train: {X_train.shape[0]} satır")
print(f"Test: {X_test.shape[0]} satır")
print(f"Hedef ortalaması – Train: {y_train.mean():.0f} TL | Test: {y_test.mean():.0f} TL")
```

Train: 41008 satır
Test: 10252 satır
Hedef ortalaması – Train: 711,552 TL | Test: 708,901 TL

8. Ön-işleme Pipeline ve Modelleme

```
In [18]: numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
])

preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features),
])

print("Preprocessor tanımlandı.")
print(f" Sayısal sütunlar ({len(numeric_features)}): {numeric_features}")
print(f" Kategorik sütunlar ({len(categorical_features)}): {categorical_features}")
```

Preprocessor tanımlandı.
Sayısal sütunlar (7): ['yil', 'km_num', 'motor_hacmi_num', 'motor_gucu_num', 'boyali_sayi', 'degisen_sayi', 'tramer_num']
Kategorik sütunlar (6): ['marka', 'vites_tipi', 'yakit_tipi', 'kasa_tipi', 'cekis', 'kimden']

```
In [19]: def evaluate_model(name, model, X_tr, y_tr, X_te, y_te):
    """Modeli eğit, test et, metrikleri döndür."""
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_te)
    mae = mean_absolute_error(y_te, y_pred)
    rmse = root_mean_squared_error(y_te, y_pred)
    r2 = r2_score(y_te, y_pred)
    print(f"name:35s} | MAE: {mae:>12,.0f} | RMSE: {rmse:>12,.0f} | R²: {r2:.4f}")
    return {"model": name, "MAE": mae, "RMSE": rmse, "R2": r2, "pipeline": model}

print("evaluate_model() tanımlandı.")
```

evaluate_model() tanımlandı.

8a. Baseline: DummyRegressor (ortalama tahmini)

```
In [20]: results = []

dummy_pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("model", DummyRegressor(strategy="mean")),
])
```

```
])
results.append(evaluate_model("Baseline (Ortalama)", dummy_pipe, X_train, y_train, X_test, y_test))
```

Baseline (Ortalama)	MAE:	374,154	RMSE:	491,977	R ² :	-0.0000
---------------------	------	---------	-------	---------	------------------	---------

8b. Ridge Regression

```
In [21]: ridge_pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("model", Ridge(alpha=1.0)),
])
results.append(evaluate_model("Ridge Regression", ridge_pipe, X_train, y_train, X_test, y_test))
```

Ridge Regression	MAE:	141,840	RMSE:	209,910	R ² :	0.8180
------------------	------	---------	-------	---------	------------------	--------

8c. ElasticNet

```
In [22]: enet_pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("model", ElasticNet(alpha=1.0, l1_ratio=0.5, max_iter=5000)),
])
results.append(evaluate_model("ElasticNet", enet_pipe, X_train, y_train, X_test, y_test))
```

ElasticNet	MAE:	170,154	RMSE:	262,973	R ² :	0.7143
------------	------	---------	-------	---------	------------------	--------

8d. Random Forest

```
In [23]: rf_pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("model", RandomForestRegressor(n_estimators=200, max_depth=20, min_samples_leaf=5,
                                   n_jobs=-1, random_state=42)),
])
results.append(evaluate_model("Random Forest", rf_pipe, X_train, y_train, X_test, y_test))
```

Random Forest	MAE:	70,718	RMSE:	114,161	R ² :	0.9462
---------------	------	--------	-------	---------	------------------	--------

8e. HistGradientBoosting (sklearn built-in, XGBoost benzeri)

```
In [24]: hgb_pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("model", HistGradientBoostingRegressor(max_iter=500, max_depth=8,
                                             learning_rate=0.05, random_state=42)),
])
results.append(evaluate_model("HistGradientBoosting", hgb_pipe, X_train, y_train, X_test, y_test))
```

HistGradientBoosting	MAE:	66,315	RMSE:	104,724	R ² :	0.9547
----------------------	------	--------	-------	---------	------------------	--------

9. Model Karşılaştırma Tablosu

```
In [25]: results_df = pd.DataFrame(results).drop(columns=["pipeline"])
results_df["MAE"] = results_df["MAE"].apply(lambda x: f"{x:,.0f}")
results_df["RMSE"] = results_df["RMSE"].apply(lambda x: f"{x:,.0f}")
results_df["R2"] = results_df["R2"].apply(lambda x: f"{x:.4f}")
results_df.set_index("model", inplace=True)
results_df
```

```
Out[25]:
```

	MAE	RMSE	R2
model			
Baseline (Ortalama)	374,154	491,977	-0.0000
Ridge Regression	141,840	209,910	0.8180
ElasticNet	170,154	262,973	0.7143
Random Forest	70,718	114,161	0.9462
HistGradientBoosting	66,315	104,724	0.9547

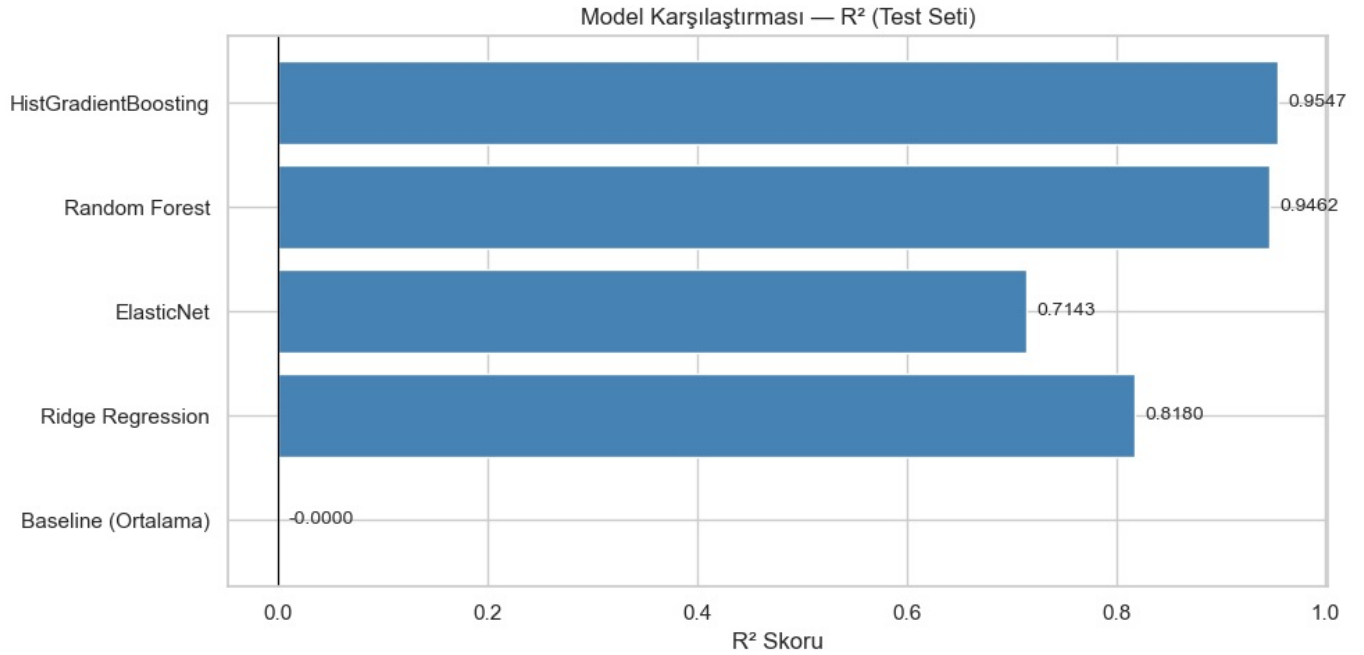
```
In [26]: r2_vals = [r["R2"] for r in results]
model_names = [r["model"] for r in results]

fig, ax = plt.subplots(figsize=(10, 5))
colors = ["grey" if v < 0 else "steelblue" for v in r2_vals]
bars = ax.barh(model_names, r2_vals, color=colors, edgecolor="white")
ax.set_xlabel("R2 Skoru")
ax.set_title("Model Karşılaştırması - R2 (Test Seti)")
ax.axvline(0, color="black", linewidth=0.8)
for bar, val in zip(bars, r2_vals):
    ax.text(bar.get_width() + 0.01, bar.get_y() + bar.get_height()/2,
```

```

        f"{val:.4f}", va="center", fontsize=10)
plt.tight_layout()
plt.show()

```



Cross-Validation: En iyi modelin (HistGradientBoosting) 5-Fold CV sonuçları

```

In [27]: best_pipe = Pipeline([
            ("preprocessor", preprocessor),
            ("model", HistGradientBoostingRegressor(max_iter=500, max_depth=8,
                                                    learning_rate=0.05, random_state=42)),
        ])

cv_r2 = cross_val_score(best_pipe, X, y, cv=5, scoring="r2", n_jobs=-1)
cv_mae = -cross_val_score(best_pipe, X, y, cv=5, scoring="neg_mean_absolute_error", n_jobs=-1)

print("5-Fold Cross-Validation Sonuçları (HistGradientBoosting):")
print(f" R² : {cv_r2.mean():.4f} ± {cv_r2.std():.4f} (fold'lar: {np.round(cv_r2, 4)})")
print(f" MAE : {cv_mae.mean():.0f} ± {cv_mae.std():.0f} (fold'lar: {np.round(cv_mae, 0)})")

```

5-Fold Cross-Validation Sonuçları (HistGradientBoosting):
 R^2 : 0.9509 ± 0.0037 (fold'lar: [0.9553 0.9515 0.9472 0.9542 0.9461])
 MAE : 67,810 ± 2,168 (fold'lar: [65646. 66931. 71699. 66256. 68519.])

10. Özellik Önemi ve Hata Analizi

```

In [28]: from sklearn.inspection import permutation_importance

best_pipe.fit(X_train, y_train)

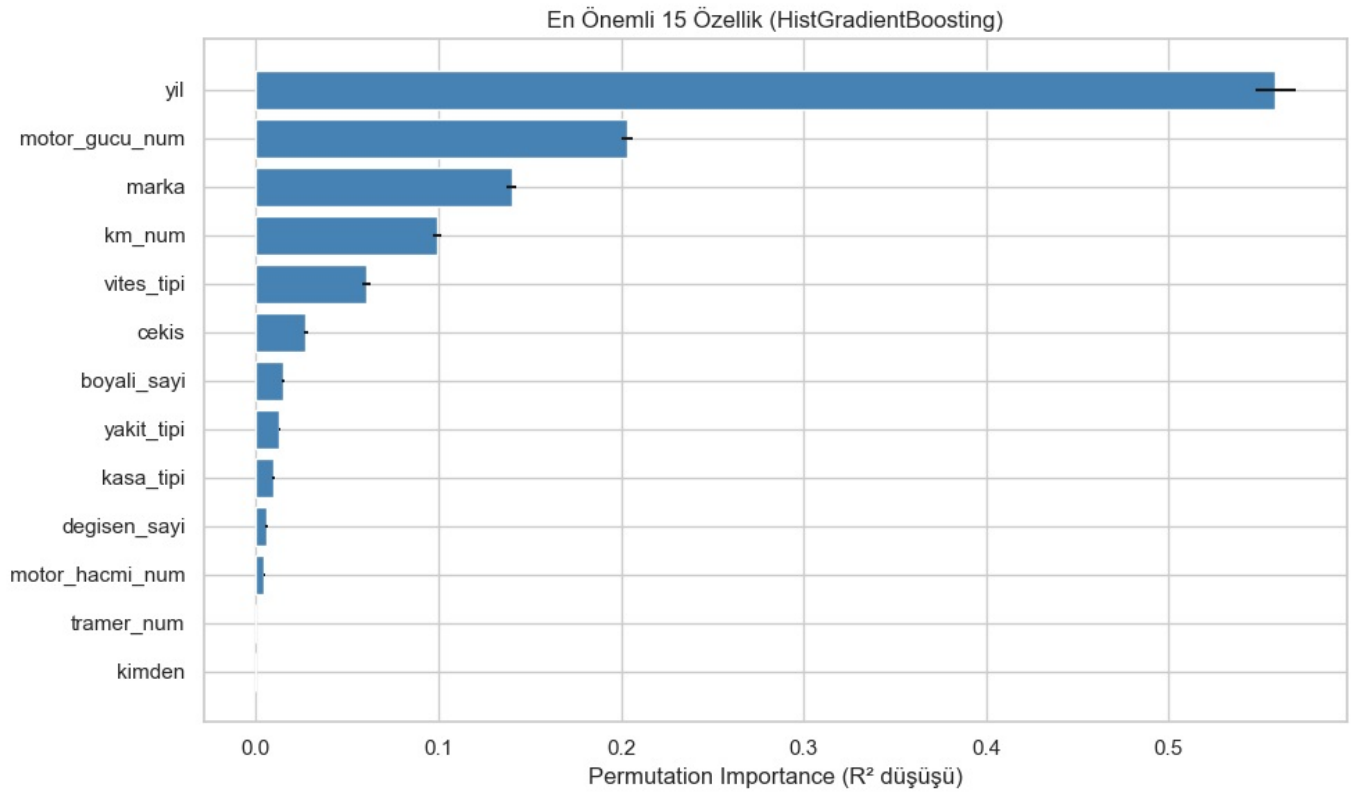
perm_imp = permutation_importance(best_pipe, X_test, y_test,
                                  n_repeats=10, random_state=42, n_jobs=-1)

feat_names = numeric_features + categorical_features
imp_df = pd.DataFrame({
    "feature": feat_names,
    "importance_mean": perm_imp.importances_mean,
    "importance_std": perm_imp.importances_std,
}).sort_values("importance_mean", ascending=False)

fig, ax = plt.subplots(figsize=(10, 6))
top_n = imp_df.head(15)
ax.barh(top_n["feature"], top_n["importance_mean"],
        xerr=top_n["importance_std"], color="steelblue", edgecolor="white")
ax.set_xlabel("Permutation Importance ( $R^2$  düşüşü)")
ax.set_title("En Önemli 15 Özellik (HistGradientBoosting)")
ax.invert_yaxis()
plt.tight_layout()
plt.show()

imp_df.head(15)

```



Out[28]:

	feature	importance_mean	importance_std
0	yil	0.558576	0.010756
3	motor_gucu_num	0.203227	0.003090
7	marka	0.140108	0.002530
1	km_num	0.099568	0.002328
8	vites_tipi	0.060430	0.002344
11	cekis	0.027394	0.001145
4	boyali_sayi	0.014742	0.000843
9	yakit_tipi	0.012968	0.000511
10	kasa_tipi	0.009711	0.000525
5	degisen_sayi	0.005739	0.000520
2	motor_hacmi_num	0.004593	0.000262
6	tramer_num	0.000053	0.000074
12	kimden	0.000017	0.000052

Hata Analizi: Gerçek vs Tahmin

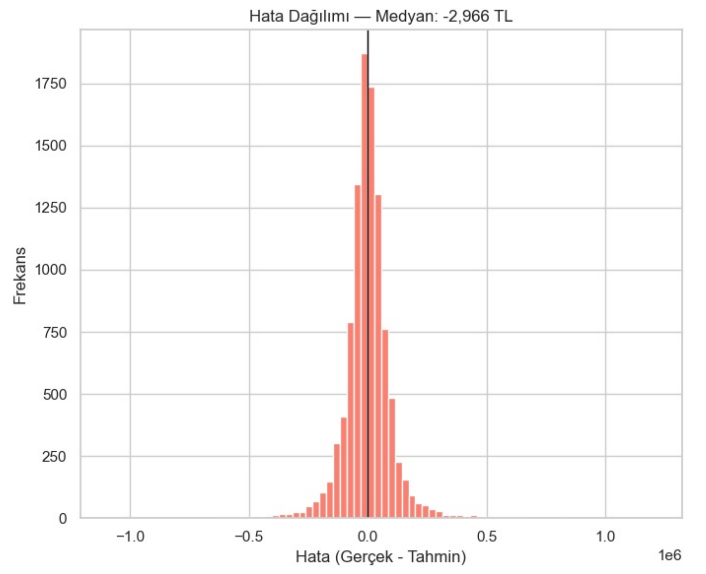
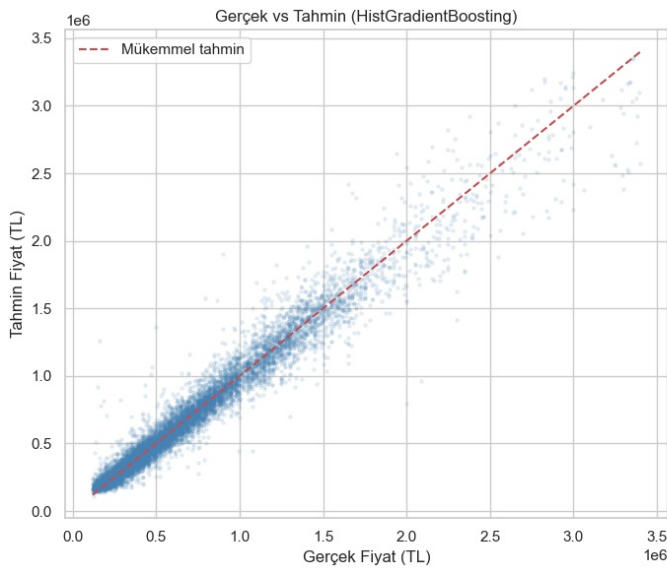
```
In [29]: y_pred_best = best_pipe.predict(X_test)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

axes[0].scatter(y_test, y_pred_best, alpha=0.1, s=5, color="steelblue")
mn, mx = y_test.min(), y_test.max()
axes[0].plot([mn, mx], [mn, mx], "r--", linewidth=1.5, label="Mükemmel tahmin")
axes[0].set_xlabel("Gerçek Fiyat (TL)")
axes[0].set_ylabel("Tahmin Fiyat (TL)")
axes[0].set_title("Gerçek vs Tahmin (HistGradientBoosting)")
axes[0].legend()

errors = y_test - y_pred_best
axes[1].hist(errors, bins=80, color="salmon", edgecolor="white")
axes[1].axvline(0, color="black", linewidth=1)
axes[1].set_xlabel("Hata (Gerçek - Tahmin)")
axes[1].set_ylabel("Frekans")
axes[1].set_title(f"Hata Dağılımı - Medyan: {np.median(errors):.0f} TL")

plt.tight_layout()
plt.show()
```



```
In [30]: error_df = X_test.copy()
error_df["gercek_fiyat"] = y_test.values
error_df["tahmin_fiyat"] = y_pred_best
error_df["hata"] = error_df["gercek_fiyat"] - error_df["tahmin_fiyat"]
error_df["abs_hata"] = error_df["hata"].abs()

print("=== En kötü 10 tahmin (en yüksek mutlak hata) ===")
worst = error_df.nlargest(10, "abs_hata")[
    ["marka", "yil", "km_num", "motor_gucu_num", "gercek_fiyat", "tahmin_fiyat", "hata"]
]

def _fmt_tl(x):
    """TL tutarını bilimsel gösterim olmadan yaz (örn. 1.234.567 TL)."""
    if pd.isna(x):
        return ""
    return f"{int(round(float(x))):,}".replace(",", ".") + " TL"

def _fmt_sayi(x):
    """km / hp gibi sayıları noktalı binlik ayırıcısıyla göster."""
    if pd.isna(x):
        return ""
    return f"{int(round(float(x))):,}".replace(",", ".")

worst_goster = worst.copy()
worst_goster["km_num"] = worst_goster["km_num"].apply(_fmt_sayi)
worst_goster["motor_gucu_num"] = worst_goster["motor_gucu_num"].apply(_fmt_sayi)
for col in ["gercek_fiyat", "tahmin_fiyat", "hata"]:
    worst_goster[col] = worst[col].apply(_fmt_tl)

worst_goster
```

=== En kötü 10 tahmin (en yüksek mutlak hata) ===

	marka	yil	km_num	motor_gucu_num	gercek_fiyat	tahmin_fiyat	hata
41063	Volkswagen	2007	320.000	225	1.999.999 TL	793.349 TL	1.206.650 TL
47656	Mercedes - Benz	1997	187.000	231	2.090.000 TL	960.450 TL	1.129.550 TL
41039	Regal Raptor	2023	6.000		165.000 TL	1.259.504 TL	-1.094.504 TL
40958	BMW	2004	167.000	272	900.000 TL	1.903.714 TL	-1.003.714 TL
43702	Volkswagen	2020	13.000	120	3.300.000 TL	2.386.544 TL	913.456 TL
42063	Mercedes - Benz	1998	389.000	238	2.300.000 TL	1.416.552 TL	883.448 TL
32884	Volvo	2002	202.000	272	500.000 TL	1.365.892 TL	-865.892 TL
46358	Audi	2019	179.000	213	3.397.500 TL	2.576.233 TL	821.267 TL
12924	Skoda	2024	15.000	138	3.325.000 TL	2.506.657 TL	818.343 TL
7388	Mercedes - Benz	2013	190.000	263	3.295.000 TL	2.485.036 TL	809.964 TL

11. Tekil Araç Fiyat Tahmini

Hemen alttaki kod hücrelerini çalıştırın. `ornek_arac` sözlüğünü kendi aracınıza göre düzenleyin. Kategorik alanlar (`marka` , `vites_tipleri` , vb.) eğitim verisinde geçen değerlerle aynı yazım olmalıdır (büyük/küçük harf dahil). Sayısal alanlar modelde

kullandığımız parse edilmiş birimlerle uyumludur: km, motor cc, hp.

```
In [43]: def fmt_tl_cikti(tutar_tl):
        """Tahmin çıktısını okunaklı TL stringine çevirir."""
        return f"{int(round(float(tutar_tl))),}".replace(",", ".") + " TL"

def arac_fiyat_tahmini(
    yıl,
    km_num,
    motor_hacmi_num,
    motor_gucu_num,
    marka,
    vites_tipi="Otomatik",
    yakit_tipi="Benzin",
    kasa_tipi="Sedan",
    cekis="Önden Çekiş",
    kimden="Galeriden",
    boyali_sayi=0.0,
    degisen_sayi=0.0,
    tramer_num=0.0,
):
    """
    Eğitilmiş best_pipe ile tahmin. Kategorik değerler veri setindeki etiketlerle aynı yazılmalı.
    """
    satir = {
        "yil": int(yıl),
        "km_num": float(km_num),
        "motor_hacmi_num": float(motor_hacmi_num) if motor_hacmi_num is not None else np.nan,
        "motor_gucu_num": float(motor_gucu_num) if motor_gucu_num is not None else np.nan,
        "boyali_sayi": float(boyali_sayi),
        "degisen_sayi": float(degisen_sayi),
        "tramer_num": float(tramer_num),
        "marka": marka,
        "vites_tipi": vites_tipi,
        "yakit_tipi": yakit_tipi,
        "kasa_tipi": kasa_tipi,
        "cekis": cekis,
        "kimden": kimden,
    }
    X_new = pd.DataFrame([satir])[numeric_features + categorical_features]
    return float(best_pipe.predict(X_new)[0])

# --- Burayı düzenleyin ---
ornek_arac = {
    "yil": 2004,
    "km_num": 185_000,
    "motor_hacmi_num": 2000,
    "motor_gucu_num": 150,
    "marka": "BMW",
    "vites_tipi": "Otomatik",
    "yakit_tipi": "LPG & Benzin",
    "kasa_tipi": "Sedan",
    "cekis": "Önden Çekiş",
    "kimden": "Galeriden",
    "boyali_sayi": 2,
    "degisen_sayi": 1,
    "tramer_num": 40000,
}

tahmin = arac_fiyat_tahmini(**ornek_arac)
print("Tahmini fiyat:", fmt_tl_cikti(tahmin))
print("\nİpucu – veri setinde geçen markalar (ilk 30):")
print(sorted(dfm["marka"].unique())[:30])
```

Tahmini fiyat: 602.216 TL

İpucu – veri setinde geçen markalar (ilk 30):

['Alfa Romeo', 'Anadol', 'Audi', 'BMW', 'BYD', 'Cadillac', 'Chery', 'Chevrolet', 'Chrysler', 'Citroen', 'Cupra', 'DS Automobiles', 'Dacia', 'Daewoo', 'Daihatsu', 'Fiat', 'Ford', 'Geely', 'Honda', 'Hyundai', 'Ikco', 'Infiniti', 'Jaguar', 'Kia', 'Lada', 'Lancia', 'MG', 'Maserati', 'Mazda', 'Mercedes - Benz']

12. Sonuç ve Değerlendirme

Proje Özeti:

- ~52.000 ikinci el araç ilanından oluşan veri seti üzerinde araç fiyat tahmini regresyon problemi çözüldü.
- 23 ham sütundan 7 sayısal + 6 kategorik özellik seçildi ve ön işleme pipeline'ı oluşturuldu.
- 5 farklı model karşılaştırıldı: Baseline, Ridge, ElasticNet, Random Forest, HistGradientBoosting.

Temel Bulgular:

- `yıl` (model yılı) ve `marka` en önemli özellikler; yeni araçlar beklendiği gibi daha pahalı.
- `kilometre` ve `motor_gucu` ikincil önemde; düşük km ve yüksek güç → yüksek fiyat.
- Ağaç tabanlı modeller (RF, HistGBT) doğrusal modellere belirgin şekilde üstün.

Sınırlamalar:

- `seri` ve `model` sütunlarının yüksek kardinalitesi nedeniyle hariç tutuldu; Target Encoding ile dahil edilmesi performansı artırabilir.
- `tramer` sütunu %90 eksik; daha zengin tramer verisi modeli güçlendirebilir.
- `konum` bilgisinden il çıkarılarak bölgesel fiyat farkları modellenebilir.
- Hiperparametre optimizasyonu (GridSearch / Optuna) yapılmadı; ek iyileştirme potansiyeli var.